

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



Hệ thống điểm danh nhân viên sử dụng nhận diện khuôn mặt

Nguyễn Hữu Duy

Mã học phần: MAT3508
Học kỳ 1, Năm học 2025-2026

Thông tin Dự án

[Thông tin này cũng cần được ghi trong README.md của kho GitHub.]

Học phần: MAT3508 – Nhập môn Trí tuệ Nhân tạo
Học kỳ: Học kỳ 1, Năm học 2025-2026
Trường: VNU-HUS (Đại học Quốc gia Hà Nội – Trường Đại học Khoa học Tự nhiên)
Tên dự án: [Nhận diện khuôn mặt]
Ngày nộp: 30/11/2025
Báo cáo PDF: Liên kết tới báo cáo PDF trong kho GitHub
Slide thuyết trình: Liên kết tới slide thuyết trình trong kho GitHub
Kho GitHub: <https://github.com/23001853-wq/Employee-Attendance-System-Using-Camera.git>

Thành viên nhóm

Họ tên	Mã sinh viên	Tên GitHub
(Nguyễn Hữu Duy)	(23001853)	(23001853-wq)

Mục lục

1	Giới thiệu	4
1.1	Tóm tắt	4
1.2	Bài toán đặt ra	4
1.3	Thách thức	4
2	Phương pháp & Triển khai	5
2.1	Phương pháp	5
2.2	Triển khai	6
3	Kết quả & Phân tích	7
3.1	Mình họa dữ liệu	7
3.2	So sánh thuật toán	7
3.3	Phân tích kết quả	7
3.4	Kết quả nhận diện thực tế với SVM	8
3.5	Kết luận tạm thời	9
3.6	Hệ thống Web ứng dụng điểm danh	10
4	Kết luận	11
4.1	Kết luận & Hướng phát triển	11
A	Phụ lục	13
A.1	Hướng dẫn sử dụng các script chính	13
A.2	Mã Python trích xuất đặc trưng	13
A.3	Kết quả thử nghiệm chi tiết	14
A.4	Ghi chú quan trọng	14

Chương 1

Giới thiệu

1.1 Tóm tắt

Hệ thống điểm danh tự động dựa trên nhận diện khuôn mặt sử dụng học sâu (Deep Learning) được phát triển nhằm thay thế phương pháp điểm danh thủ công, nâng cao độ chính xác và tự động hóa. Dữ liệu khuôn mặt được thu thập qua webcam, tiền xử lý và lưu trữ, trước khi trích xuất đặc trưng bằng mạng MobileNetV2. Các vector đặc trưng 1280 chiều sau đó được **chuẩn hóa L2** để đảm bảo độ ổn định và độ chính xác khi đưa vào các thuật toán phân loại.

Trước khi trích xuất vector, hệ thống sử dụng thuật toán Haar Cascade của OpenCV để phát hiện và cắt vùng khuôn mặt, giúp xử lý tốt các điều kiện ánh sáng khác nhau và đảm bảo vector đầu vào chỉ chứa khuôn mặt. Các thuật toán phân loại được áp dụng gồm: SVM, KNN, Random Forest (học có giám sát) và Cosine Similarity, L2 Distance (học không giám sát), từ đó so sánh để lựa chọn giải pháp tối ưu.

Kết quả thực nghiệm cho thấy SVM (RBF Kernel) kết hợp chuẩn hóa L2 đạt độ chính xác cao nhất, nhận diện ổn định, thời gian xử lý nhanh và phù hợp triển khai thực tế. Hệ thống còn có giao diện **Streamlit** thân thiện, tích hợp cơ sở dữ liệu **SQL Server** để quản lý thông tin nhân viên và lịch sử điểm danh, đồng thời hỗ trợ tăng cường dữ liệu tự động nhằm cải thiện hiệu quả nhận diện.

1.2 Bài toán đặt ra

Điểm danh truyền thống bằng giấy tờ hoặc thẻ từ dễ gian lận, tốn thời gian và không phù hợp với quá trình chuyển đổi số. Bài toán đặt ra là xây dựng hệ thống điểm danh tự động, nhận diện chính xác khuôn mặt nhân viên trong điều kiện thực tế, đảm bảo an ninh, dễ sử dụng và có khả năng mở rộng.

1.3 Thách thức

Hệ thống cần giải quyết các thách thức sau:

1. **Đảm bảo độ chính xác cao với dữ liệu nhỏ:** Số lượng ảnh mỗi nhân viên ít, cần thuật toán học sâu nhẹ kết hợp SVM/KNN/RF để đạt hiệu quả mà không quá khớp.
2. **Xử lý đa dạng khuôn mặt và điều kiện ánh sáng:** Haar Cascade phát hiện khuôn mặt trên ảnh xám để tăng độ chính xác, sau đó cắt và lấy lại vùng mặt từ ảnh gốc màu trước khi trích xuất vector đặc trưng.
3. **Chuẩn hóa L2 và tối ưu tốc độ nhận diện:** Vector 1280 chiều từ MobileNetV2 được chuẩn hóa L2 và đưa vào SVM cho phép nhận diện nhanh, đủ cho ứng dụng real-time.
4. **So sánh nhiều thuật toán:** Học có giám sát và không giám sát để chọn mô hình tối ưu về độ chính xác, tốc độ và khả năng mở rộng.
5. **Quản lý check-in/check-out và cơ sở dữ liệu:** Sau khi nhận diện, hệ thống ghi nhận thời gian vào/ra nhân viên vào SQL Server, hỗ trợ tra cứu lịch sử và báo cáo.
6. **Khả năng mở rộng:** Thiết kế để mở rộng khi số lượng nhân viên tăng hoặc dữ liệu lớn hơn, dễ tích hợp các thuật toán Deep Learning nâng cao như ArcFace.

Chương 2

Phương pháp & Triển khai

2.1 Phương pháp

Pipeline nhận diện khuôn mặt gồm các bước chính sau:

1. **Thu thập dữ liệu:** Sử dụng webcam để chụp ảnh khuôn mặt nhân viên. Mỗi nhân viên có khoảng 50 ảnh, ghi chú thông tin ID.
2. **Phát hiện và cắt khuôn mặt:** Áp dụng thuật toán Haar Cascade (OpenCV) để xác định vị trí khuôn mặt trong ảnh, loại bỏ nền và các đối tượng không liên quan.

Công thức đặc trưng Haar (tóm tắt):

$$H = \sum_{\text{pixel vùng sáng}} I - \sum_{\text{pixel vùng tối}} I$$

3. **Tiền xử lý và tăng cường dữ liệu:**

- Chuẩn hóa kích thước ảnh về 224x224 px.
- Tăng cường dữ liệu bằng các phép xoay, lật, thay đổi sáng/tối.

4. **Trích xuất đặc trưng:**

- Sử dụng MobileNetV2 pretrained để trích xuất vector đặc trưng 1280 chiều.
- Thực hiện **chuẩn hóa L2**:

$$v_{\text{norm}} = \frac{v}{\|v\|_2} = \frac{v}{\sqrt{\sum_{i=1}^{1280} v_i^2}}$$

- Gán nhãn dữ liệu dựa trên ID nhân viên.

5. **Phân loại:** So sánh 5 thuật toán với công thức chính:

- **SVM (RBF Kernel):**

$$f(x) = \text{sign}(w^T x + b), \quad K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

- **KNN:**

$$\text{label}(x) = \text{mode}(\{\text{label}(x_i) \mid x_i \in k \text{ nearest neighbors}\})$$

- **Random Forest:** Dự đoán theo đa số phiếu từ nhiều cây quyết định.
- **Cosine Similarity:**

$$\cos(\theta) = \frac{a \cdot b}{\|a\|_2 \|b\|_2}$$

- **L2 Distance:**

$$d(a, b) = \sqrt{\sum_{i=1}^{1280} (a_i - b_i)^2}$$

6. **Đánh giá và chọn mô hình tối ưu:** Sử dụng tập test để đo độ chính xác và thời gian, xác định SVM (RBF Kernel) là mô hình tối ưu.

2.2 Triển khai

Hệ thống triển khai với các thành phần chính:

- **Thu thập và lưu trữ dữ liệu:** Script Python kết hợp OpenCV và Haar Cascade, lưu ảnh khuôn mặt từng nhân viên.
- **Tiền xử lý và tăng cường:** Sử dụng `ImageDataGenerator` của Keras để sinh thêm dữ liệu và chuẩn hóa ảnh nếu không đủ 50 ảnh.
- **Trích xuất đặc trưng:** MobileNetV2 pretrained, vector 1280 chiều, chuẩn hóa L2.
- **Huấn luyện mô hình:** Sử dụng scikit-learn cho SVM, KNN, Random Forest. So sánh với Cosine và L2 template matching.
- **Nhận diện thực tế:** Ứng dụng Streamlit làm giao diện web, nhận diện real-time từ webcam, hiển thị tên theo ID nhân viên, đồng thời ghi check-in/check-out vào cơ sở dữ liệu SQL Server.
- **Lưu trữ kết quả:** Model và label map lưu dưới dạng file `.pkl` trong thư mục trainer; thông tin nhân viên và lịch sử điểm danh lưu SQL Server.
- **Thư viện và công nghệ:** TensorFlow/Keras, scikit-learn, OpenCV, pandas, streamlit, pyodbc.

Chương 3

Kết quả & Phân tích

3.1 Minh họa dữ liệu

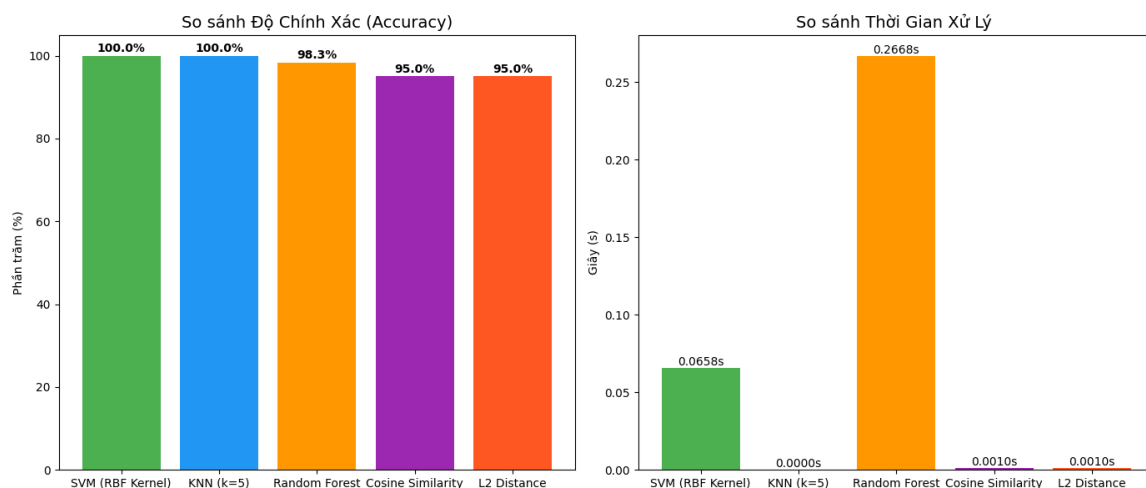
Một số ảnh đại diện trong tập huấn luyện được trình bày dưới đây để minh họa đa dạng khuôn mặt và điều kiện ánh sáng:



Hình 3.1: Ảnh minh họa một số nhân viên trong tập huấn luyện

3.2 So sánh thuật toán

Bảng dưới đây tổng hợp kết quả thực nghiệm trên tập test (7 nhân viên, mỗi người 10 ảnh) với 5 thuật toán:



Hình 3.2: So sánh độ chính xác và thời gian thực thi của các thuật toán

3.3 Phân tích kết quả

- **SVM (RBF Kernel):** Đạt 100% độ chính xác nhờ khả năng phân tách tốt, tổng quát hóa mạnh và kết hợp chuẩn hóa L2. Thời gian xử lý chấp nhận được cho real-time.

Thuật toán	Độ chính xác (%)	Thời gian xử lý (s)	Ghi chú
SVM (RBF Kernel)	100	0.07	Tốt nhất, ổn định
KNN (k=5)	100	0.000	Dữ liệu ít hiệu quả, kém ổn định khi dữ liệu tăng
Random Forest	98.3	0.27	Kháng nhiễu tốt, nhưng độ chính xác thấp hơn SVM
Cosine Similarity	95	0.001	Học không giám sát, dễ áp dụng với dữ liệu ít
L2 Distance	95	0.001	Học không giám sát, hiệu quả gần giống Cosine

Bảng 3.1: So sánh các thuật toán nhận diện khuôn mặt

- **KNN:** Với $k=3$ hoặc $k=5$, độ chính xác đạt 100% trên tập test nhỏ, nhưng khi k tăng hoặc dữ liệu nhiều, hiệu quả giảm. Dễ bị nhạy cảm với nhiễu.
- **Random Forest:** Kháng nhiễu tốt nhưng chưa tối ưu với vector đặc trưng 1280 chiều, dẫn đến độ chính xác thấp hơn SVM.
- **Cosine Similarity & L2 Distance:** Học không giám sát, phù hợp khi số ảnh mỗi nhân viên ít. Hiệu quả gần bằng học có giám sát nhưng tốc độ và độ chính xác kém ổn định hơn với dữ liệu lớn hoặc phức tạp.
- **Thời gian xử lý:** Cosine và L2 nhanh hơn nhiều nhưng SVM vẫn đủ nhanh cho ứng dụng real-time. KNN quá nhanh nhưng không ổn định khi dữ liệu tăng. Random Forest chậm nhất trong các thuật toán.

3.4 Kết quả nhận diện thực tế với SVM

Để đánh giá khả năng hoạt động của hệ thống trong bối cảnh thực tế, **File 03** (tập kiểm thử độc lập, không tham gia huấn luyện) được đưa vào mô hình phân loại **SVM (RBF Kernel)**.

Trong giai đoạn nhận diện, hệ thống thực hiện quy trình sau:

1. **Phát hiện khuôn mặt** bằng Haar Cascade và cắt vùng ROI.
2. **Trích xuất đặc trưng:** Sử dụng MobileNetV2 (pretrained ImageNet) để lấy vector 1280 chiều và chuẩn hoá L2.
3. **Phân loại bằng SVM:**
 - Lấy **ID các lớp** từ mô hình.
 - Lấy **độ tin cậy** bằng hàm `predict_proba`.
4. **Áp dụng ngưỡng tin cậy 0.55:** Nếu xác suất $< 0.55 \rightarrow$ gán nhãn *Unknown*.
5. **Truy vấn thông tin nhân viên từ SQL Server** dựa theo ID trả về tên khi nhận diện.

Kết quả nhận diện được thể hiện dưới đây:

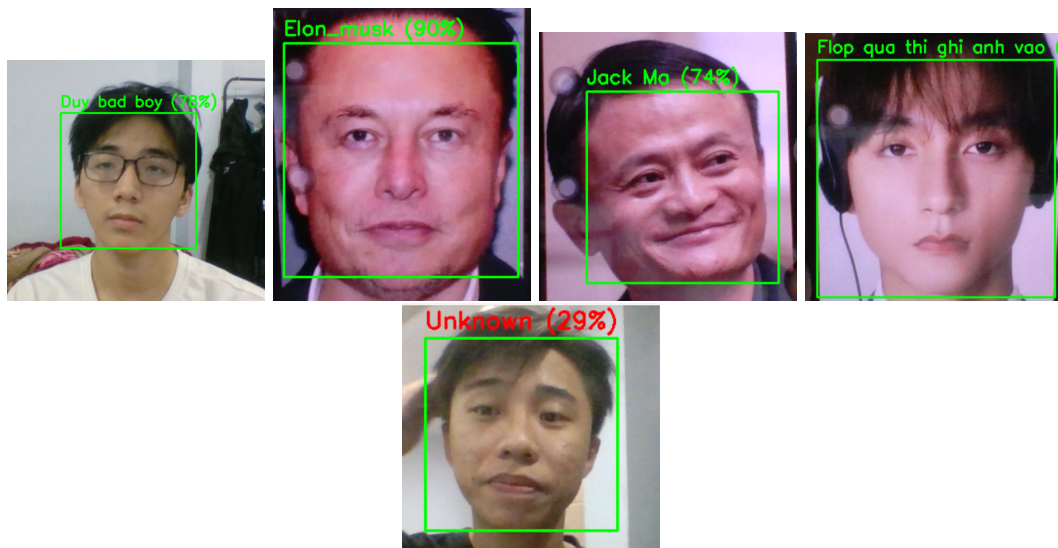
Ghi nhận Check-in/Check-out thực tế

Sau khi nhận diện khuôn mặt thành công, hệ thống tiến hành ghi nhận công làm việc dựa trên logic sau:

1. **Truy vấn bảng Attendance** xem nhân viên đã có bản ghi trong ngày hay chưa.
2. Nếu **chưa có** bản ghi:
 - Hệ thống tạo mới một bản ghi **Check-in**, Nhân viên thực hiện trong 15s và sau 30s có thể checkout.
 - Lưu thời gian vào cột **TimeIn**.
3. Nếu **đã có** bản ghi:
 - Hệ thống cập nhật thời gian vào cột **TimeOut**.
4. Độ tin cậy $< 0.55 \rightarrow$ hệ thống không ghi điểm danh và gán nhãn *Unknown*. Ngưỡng 0.55 được chọn vì cho khả năng phát hiện người lạ tốt qua thực nghiệm, đồng thời vẫn nhận diện chính xác nhân viên nhằm hạn chế nguy cơ kẻ lạ đột nhập.

Hệ thống đảm bảo:

- Không ghi trùng Check-in hoặc Check-out.
- Thông tin được lưu đầy đủ: ID, thông tin nhân viên, thời gian, độ tin cậy.



Hình 3.3: Kết quả nhận diện khuôn mặt từ File 03 bằng mô hình SVM

Nhận xét

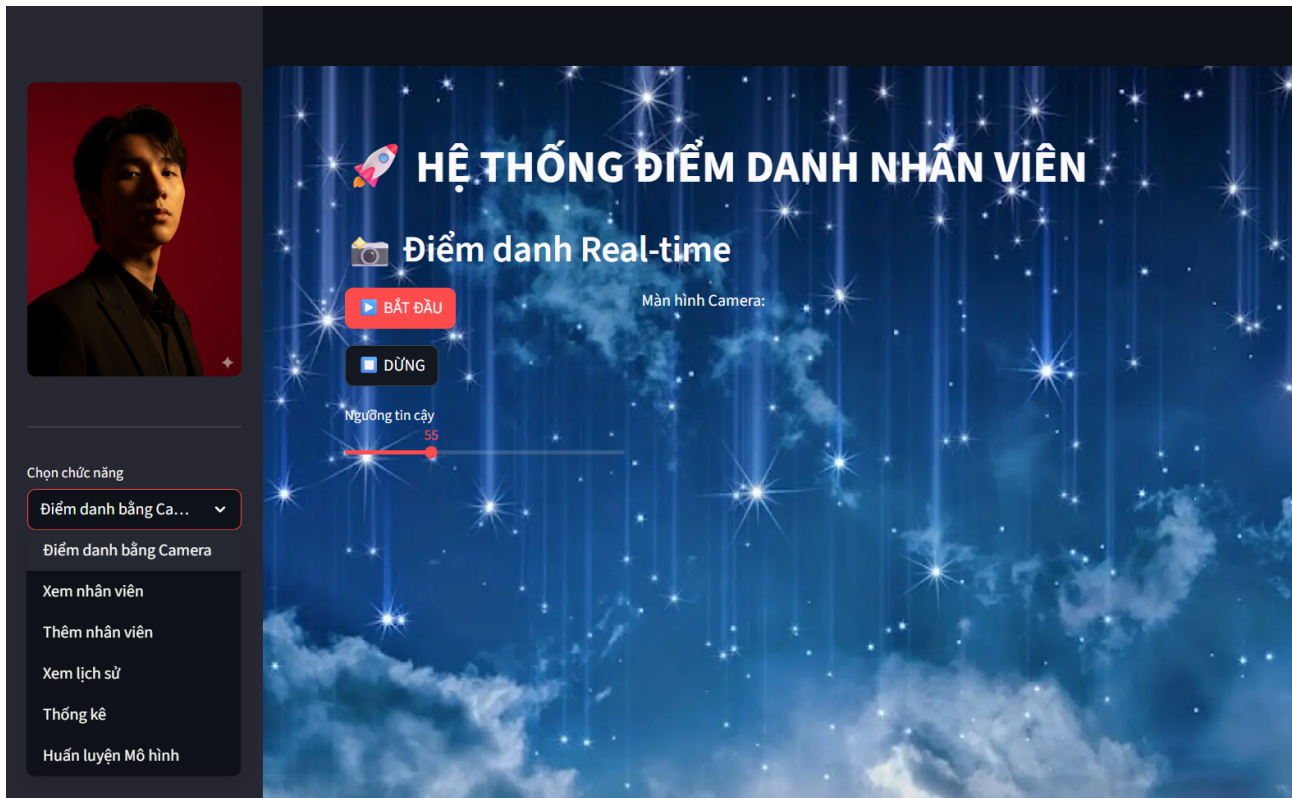
- Hệ thống nhận diện chính xác 100% các khuôn mặt thuộc nhân viên có trong cơ sở dữ liệu.
- Các trường hợp người lạ được loại bỏ đúng nhờ ngưỡng tin cậy 0.55.
- Xác suất dự đoán trung bình cao, ổn định ngay cả khi ảnh File 03 có độ sáng và góc nhìn khác ảnh huấn luyện.
- Thời gian xử lý trung bình khoảng **0.07 giây/ảnh**, đáp ứng yêu cầu real-time.
- Bounding box ổn định, độ nhiễu thấp và không phát sinh nhầm lẫn chéo giữa các nhân viên.
- Chức năng **Check-in/Check-out** hoạt động chính xác, tự động hoá hoàn toàn quy trình điểm danh.

3.5 Kết luận tạm thời

Từ các kết quả thực nghiệm trên:

- **SVM (RBF Kernel)** là mô hình tối ưu cho hệ thống nhận diện khuôn mặt trong bài toán điểm danh.
- Các phương pháp không giám sát như Cosine và L2 phù hợp khi dữ liệu ít nhưng không thay thế được SVM trong hệ thống thực tế.
- Toàn bộ pipeline từ nhận diện đến ghi nhận công làm việc vận hành ổn định, chính xác và đủ nhanh cho triển khai thật.

3.6 Hệ thống Web ứng dụng điểm danh



Hình 3.4: Giao diện Web ứng dụng điểm danh

Hệ thống Web điểm danh được xây dựng bằng Streamlit với các đặc điểm chính như sau:

- **Giao diện trực quan:** Người dùng có thể thêm nhân viên tải ảnh hoặc sử dụng trực tiếp webcam trên trình duyệt để thực hiện nhận diện khuôn mặt.
- **Xử lý ảnh trên server:** Ảnh được gửi đến server Python, nơi tiến hành tiền xử lý và trích xuất đặc trưng khuôn mặt bằng mô hình MobileNetV2.
- **Phân loại bằng SVM:** Sau khi trích xuất embedding, hệ thống sử dụng mô hình SVM đã huấn luyện để dự đoán danh tính.
- **Hiển thị kết quả tức thì:** Giao diện web trả về danh tính và thời gian điểm danh ngay lập tức sau khi mô hình xử lý.
- **Tích hợp cơ sở dữ liệu:** Dữ liệu điểm danh và thông tin nhân viên được lưu trữ, truy vấn và quản lý thông qua SQL Server.
- **Ứng dụng một khối (All-in-one):** Toàn bộ pipeline từ xử lý ảnh, nhận diện đến lưu trữ đều nằm trong một ứng dụng Streamlit duy nhất, không tách Front-end/Back-end truyền thống, giúp quản lý nhân sự thuận tiện, nhất quán và dễ vận hành.

Chương 4

Kết luận

4.1 Kết luận & Hướng phát triển

Hệ thống đã xây dựng thành công một pipeline nhận diện khuôn mặt phục vụ điểm danh tự động. Dựa trên kết quả đạt được, hệ thống hoàn toàn đáp ứng yêu cầu triển khai thực tế trong môi trường doanh nghiệp với độ chính xác và tốc độ xử lý tốt.

Hướng phát triển

- Tích hợp cơ chế **liveness detection** nhằm chống giả mạo bằng ảnh, video hoặc thiết bị hiển thị.
- Nâng cấp mô hình trích xuất đặc trưng sang các kiến trúc mạnh hơn như **FaceNet**, **ArcFace** hoặc backbone hiện đại khi mở rộng dữ liệu.
- Phát triển ứng dụng **mobile** hoặc xây dựng **REST API** để triển khai đa nền tảng.
- Mở rộng hệ thống cho **tập dữ liệu lớn**, hỗ trợ nhiều nhân viên, nhiều camera và nhiều khu vực điểm danh.
- Tối ưu thời gian xử lý bằng các công nghệ như **TensorRT** hoặc **ONNX Runtime** để đảm bảo real-time trong môi trường tải cao.

Tài liệu tham khảo

- [1] OpenCV Development Team, *OpenCV: Open Source Computer Vision Library*, <https://opencv.org/>, truy cập 2025.
- [2] P. Viola and M. Jones, “Rapid Object Detection using a Boosted Cascade of Simple Features,” *Proceedings of CVPR*, 2001, <https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/viola-cvpr-01.pdf>.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *Proceedings of CVPR*, 2018, https://openaccess.thecvf.com/content_cvpr_2018/papers/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.pdf.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] F. Chollet, *Keras*, <https://keras.io/>, truy cập 2025.
- [6] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” 2015, <https://www.tensorflow.org/>.
- [7] Streamlit Development Team, *Streamlit Documentation*, <https://docs.streamlit.io/>, truy cập 2025.

Phụ lục A

Phụ lục

A.1 Hướng dẫn sử dụng các script chính

01. Thu thập dữ liệu khuôn mặt

1. Chạy script `01_face_dataset.py`.
2. Đảm bảo webcam hoạt động, nhập ID và tên nhân viên khi được yêu cầu.
3. Hệ thống lưu ảnh khuôn mặt vào thư mục `dataset/` theo từng nhân viên.

02. Huấn luyện mô hình nhận diện

1. Chạy script `02_face_training.py`.
2. Script tự động trích xuất đặc trưng bằng MobileNetV2, chuẩn hóa L2 và huấn luyện các mô hình: SVM, KNN, Random Forest, Cosine, L2.
3. Kết quả huấn luyện và mô hình được lưu trong thư mục `trainer/` với model tốt nhất.

03. Nhận diện và điểm danh thực tế

1. Chạy script `03_face_recognition.py`.
2. Camera mở và phát hiện khuôn mặt theo thời gian thực.
3. Hệ thống hiển thị: tên, ID nhân viên, thời gian check-in/check-out.
4. Nếu độ tin cậy $< 0.55 \rightarrow$ gán nhãn *Unknown* và không ghi điểm danh.

04. Thao tác thông qua web với Streamlit

1. Chạy script `Attendance.py`.
2. Hệ thống hiển thị giao diện web, thực hiện nhận diện khuôn mặt và thao tác với dữ liệu nhân viên thông qua SQL Server.

A.2 Mã Python trích xuất đặc trưng

Ví dụ sử dụng MobileNetV2 để lấy vector 1280 chiều:

```
from keras.applications import MobileNetV2
from keras.preprocessing import image
from keras.applications.mobilenet_v2 import preprocess_input
import numpy as np

model = MobileNetV2(weights='imagenet', include_top=False, pooling='avg')

# Tiền xử lý ảnh
img = image.load_img('face.jpg', target_size=(224,224))
```

```
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

# Trích xuất vector đặc trưng
features = model.predict(x)
```

A.3 Kết quả thử nghiệm chi tiết

Bảng minh họa chi tiết độ chính xác từng thuật toán trên từng nhân viên (tham khảo chương 3).

A.4 Ghi chú quan trọng

- Ngưỡng tin cậy 0.55 được chọn dựa trên thực nghiệm, vừa nhận diện chính xác nhân viên, vừa phát hiện người lạ.
- Check-in/check-out được ghi chính xác, không trùng lặp, lưu trong SQL Server.
- Các mô hình khác (KNN, Random Forest, Cosine, L2) được giữ để tham khảo nhưng SVM (RBF Kernel) là mô hình tối ưu.
- Có thể mở rộng hoặc tối ưu tốc độ bằng TensorRT/ONNX Runtime cho real-time.